

Lista 2

SZTUCZNA INTELIGENCJA

KAMIL TATROCKI (280506)

opis teoretyczny metody

Algorytm Minimax jest rekurencyjną metodą przeszukiwania drzewa gry w głąb. Działa on na zasadzie wstecznej indukcji: najpierw schodzi do węzłów końcowych lub osiąga zadany limit głębokości, a następnie propaguje wartości ocen w górę drzewa. W procesie tym rozróżniamy dwa rodzaje węzłów: maksymalizujące (reprezentujące ruch gracza) oraz minimalizujące (reprezentujące ruch przeciwnika). Gracz maksymalizujący wybiera spośród dostępnych ruchów ten, który prowadzi do stanu o najwyższej wartości, natomiast gracz minimalizujący wybiera ruch o wartości najniższej.

```
PS C:\github\AI\breakthrough> uv run main.py
Tryb gry [basic/extended]:
Algorytm [1-AlphaBeta/2-Minimax]: 2
Wymiary planszy [n m]:
Głębokość Agent B:
Heurystyka B [1-mat/2-pos/3-mix]:
Głębokość Agent W:
Heurystyka W [1-mat/2-pos/3-mix]:
Własna plansza? [t/N]:
=== KOŃCOWY STAN PLANSZY ===
  A B C D E F G H
1 W _ _ _ _ B B B
2 o _ _ B B B B B
3 B _ B B _ _ _ _
4 B B _ _ _ _ _ _
5 _ _ _ _ _ _ _ _
6 _ _ _ _ _ _ _ _
7 _ _ _ _ W W W W
8 W W W W W W W W

Rundy:      33
Zwycięzca: Biały (W)

[STATYSTYKI]
Odwiedzone węzły: 432066
Czas gry:      34.8556 s
Tryb:          extended
Agent B: głębokość=3, heurystyka=defensive_aggressive_mix, algorytm=Minimax
Agent W: głębokość=3, heurystyka=material_advantage, algorytm=Minimax
PS C:\github\AI\breakthrough> 
```

Algorytm Alpha-Beta jest optymalizacją metody Minimax, która pozwala na znaczne ograniczenie liczby odwiedzanych węzłów (parametr `nodes_visited`), nie wpływając przy tym na ostateczny wynik obliczeń. Główną ideą jest pominięcie tych gałęzi drzewa, o których na podstawie już sprawdzonych ścieżek wiadomo, że nie mogą wpłynąć na finalną decyzję gracza.

```

PS C:\github\AI\breakthrough> uv run main.py
Tryb gry [basic/extended]:
Algorytm [1-AlphaBeta/2-Minimax]: 1
Wymiary planszy [n m]:
Głębokość Agent B:
Heurystyka B [1-mat/2-pos/3-mix]:
Głębokość Agent W:
Heurystyka W [1-mat/2-pos/3-mix]:
Własna plansza? [t/N]:
=== KOŃCOWY STAN PLANSZY ===
  A B C D E F G H
1 W _ _ _ _ B B B
2 o _ _ B B B B B
3 B _ B B _ _ _ _
4 B B _ _ _ _ _ _
5 _ _ _ _ _ _ _ _
6 _ _ _ _ _ _ _ _
7 _ _ _ _ W W W W
8 W W W W W W W W

Rundy:      33
Zwycięzca: Biały (W)

[STATYSTYKI]
Odwiedzone węzły: 81687
Czas gry:      7.0269 s
Tryb:          extended
Agent B: głębokość=3, heurystyka=defensive_aggressive_mix, algorytm=Alpha-Beta
Agent W: głębokość=3, heurystyka=material_advantage, algorytm=Alpha-Beta
PS C:\github\AI\breakthrough>

```

formalne sformułowanie problemu z ograniczeniami

Problem został zdefiniowany jako zadanie przeszukiwania przestrzeni stanów w grze dwuosobowej o sumie zerowej, rozgrywanej na prostokątnej planszy o wymiarach $n \times m$. Formalnie model ten można opisać jako krotkę składającą się ze zbioru możliwych konfiguracji planszy, zestawu dopuszczalnych przejść oraz jasno określonych warunków końcowych. Stan początkowy reprezentuje układ, w którym dwa pierwsze i dwa ostatnie rzędy planszy są w całości wypełnione pionami odpowiednio gracza czarnego (B) i białego (W). Przestrzeń stanów obejmuje wszystkie możliwe układy pionów, jakie mogą powstać w wyniku naprzemiennych ruchów obu stron, przy czym każdy element planszy może przyjmować jedną z trzech wartości: puste pole, pion czarny lub pion biały.

Zbiór akcji dla każdego stanu jest ściśle ograniczony przez zasady poruszania się figur. Dla konkretnego gracza zbiór dostępnych ruchów w danym stanie zależy od aktualnego rozmieszczenia pionów oraz kierunku przypisanego do koloru. Ruch definiowany jest jako para współrzędnych. Przejście między stanami następuje poprzez funkcję przejścia, która aktualizuje zawartość pól na planszy: pole źródłowe staje się puste, a pole docelowe przyjmuje wartość gracza wykonującego ruch, co ewentualnie skutkuje usunięciem piona przeciwnika.

Ograniczenia i reguły przejścia

Dopuszczalność ruchu w systemie podlega restrykcyjnym ograniczeniom logicznym i fizycznym planszy. Po pierwsze, każda akcja musi mieścić się w granicach wyznaczonych przez parametry n i m , co oznacza, że współrzędne wierszy muszą zawierać się w przedziale $[0, n-1]$, a kolumn w

przedziale $[0, m-1]$. Po drugie, piony mogą poruszać się wyłącznie w jednym kierunku wzdłuż osi wierszy: piony czarne zwiększają indeks wiersza, natomiast białe go zmniejszają.

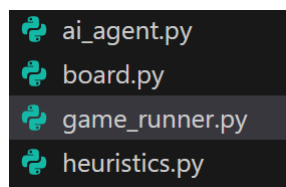
Dodatkowe ograniczenia dotyczą interakcji z polami docelowymi. Ruch pionowo do przodu (w tej samej kolumnie) jest dozwolony wyłącznie wtedy, gdy pole docelowe jest puste. Ruchy skośne (o jedną kolumnę w lewo lub w prawo) są dopuszczalne zarówno na pola puste, jak i na pola zajęte przez przeciwnika, co stanowi jedyny sposób eliminacji pionów drugiej strony. System uniemożliwia wykonanie ruchu na pole zajęte przez własny pion oraz wykonanie ruchu z pola, które nie zawiera piona aktywnego gracza.

Funkcja celu i warunki zakończenia

Cel gry zostaje osiągnięty, gdy spełniony zostanie jeden z warunków testu stanu końcowego. Pierwszym kryterium jest dotarcie dowolnym pionem do ostatniego rzędu po stronie przeciwnika – dla gracza czarnego jest to wiersz $n-1$, a dla białego wiersz 0. Drugim kryterium zakończenia jest sytuacja patowa z perspektywy jednego z graczy, w której nie posiada on żadnych legalnych ruchów do wykonania. W takim przypadku zwycięstwo przypisywane jest stronie przeciwnej. Z matematycznego punktu widzenia, algorytmy sterujące dążą do maksymalizacji wartości funkcji celu, która w stanach terminalnych przyjmuje wartości ekstremalne, a w stanach pośrednich jest szacowana przez zdefiniowane heurystyki oceniające przewagę materialną i pozycję na planszy.

opis idei rozwiązania

Kod został podzielony na 4 klasy



Game Runner – zawiera agentów oraz obecny board

Board – zawiera informację o aktualnej planszy (ważne, stan gry jest immutable)

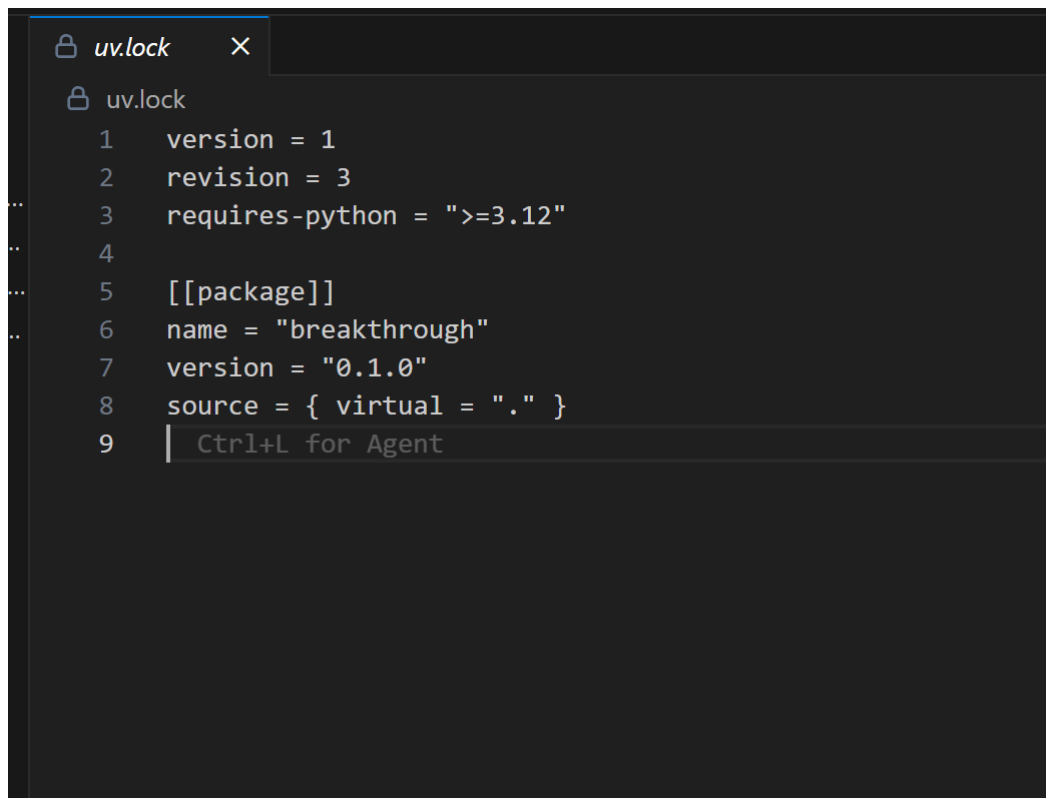
AI_agent - instancja zawodnika z białymi i czarnymi pionkami

Heuristics – metody heurystyczne spełniające ogólny interfejs

wykorzystane materiały źródłowe

Prezentacja profesora Macieja Piaseckiego pt. „Gry logiczne”.

biblioteki wykorzystane przy implementacji



```
uv.lock
1 version = 1
2 revision = 3
3 requires-python = ">=3.12"
4
5 [[package]]
6 name = "breakthrough"
7 version = "0.1.0"
8 source = { virtual = "." }
9 | Ctrl+L for Agent
```

Nie wykorzystuje żadnych zewnętrznych bibliotek jak widać, tylko to co oferuje podstawowy python.

napotkane problemy implementacyjne przy wykonywaniu zadania

Nie napotkałem na żadne problemy, implementacja tej listy była dla stosunkowo prosta.